

CampusClaw

身份认证与知识库问答 · 实现方案

基于现有代码梳理：token 方案登录认证 / 知识库检索与引用标注问答

项目	内容
文档主题	结合现有代码梳理「基于 token 的身份认证」与「基于知识库的问答」两条链路的实现方案
代码仓库	https://github.com/OhtoAi583/campusclaw
对应迭代	迭代 1（认证授权与知识库入库）、迭代 2（限班级范围检索与内容溯源）、课堂任务（token 认证、知识库问答）
技术栈	Go（net/http，不引入 Web 框架）+ MySQL 8.0 + React 18 + Nginx + Docker Compose；回答生成接 DeepSeek 对话网关
验证方式	后端单元测试 + 验收脚本 + 浏览器实测截图（截图与命令输出均取自真实运行结果）

1. 系统概览

CampusClaw 是面向中小学教研场景的知识库底座。请求链路只有一条：浏览器 → Nginx → Go → MySQL；浏览器与静态页面一律不可信，认证、授权、班级隔离、检索与生成全部发生在 Go 服务内。数据库只在 Compose 内部网络可达，上传目录只挂给 api 容器，不经 Nginx 静态暴露。

本文梳理两条主线：

- ① 基于 token 的身份认证：账号口令换取 JWT 访问令牌，受保护接口用 **Authorization: Bearer** 携带令牌；
- ② 基于知识库的问答：材料入库并切分为带字符区间的切片，提问时先在本班范围内检索，取得切片之后才调用生成模型，回答中用 [1][2] 标注出处。

层	选型	职责
前端	React 18 + TypeScript + Vite	登录页、材料页、检索页、问答演示页；只调用同源 /api
入口	Nginx	托管前端构建产物；反代 /api；上传目录显式返回 404
服务	Go + net/http	认证与授权、班级隔离、上传与入库、检索、回答生成
存储	MySQL 8.0	用户、会话、刷新令牌、材料、知识库正文、切片与向量
外部	DeepSeek 对话网关	仅在检索到切片后调用，用于生成带出处的回答

2. 基于 token 的身份认证

2.1 设计目标

token 方案与迭代 1 的服务端会话 (Cookie) 并存，两者最终换回同一组身份 (user_id / role / class_id)，因此授权与班级隔离的逻辑只有一份。设计时坚持三条：

1. 令牌载荷不带权限声明，role 与 class_id 每次回库读取；
2. 访问令牌短期有效，刷新令牌可轮换、可撤销；
3. 签名密钥只存在于服务端环境变量，缺失时服务拒绝启动。

2.2 令牌结构

访问令牌是 HS256 签名的紧凑 JWT，三段式 header.payload.signature。载荷刻意只保留 sub (用户 id) 与 jti、iat、exp：载荷是客户端可读的，把权限声明写进去容易被误当成授权依据。

代码 2-1 令牌载荷定义 (backend/internal/auth/token.go)

```
type TokenClaims struct {
    Subject    int64  `json:"sub"`
    TokenID    string `json:"jti"`
    IssuedAt   int64  `json:"iat"`
    ExpiresAt  int64  `json:"exp"`
}
```

2.3 签发：账号口令换令牌

登录接口先按用户名查库并做 bcrypt 校验 (未知用户也会执行一次等成本的假校验，避免通过响应时间区分账号是否存在)，通过后签发一对令牌：JWT 访问令牌 +

随机刷新令牌。刷新令牌只有哈希入库，因此可以随时撤销。

代码 2-2 签发访问令牌与刷新令牌 (backend/internal/auth/token.go)

```
func (t *TokenService) Issue(ctx context.Context, userID int64) (TokenPair, error) {
    now := t.now()
    claims := TokenClaims{
        Subject:    userID,
        TokenID:    randomHex(16),
        IssuedAt:   now.Unix(),
        ExpiresAt:  now.Add(t.accessTTL).Unix(),
    }
    access, err := t.sign(claims)
    if err != nil {
        return TokenPair{}, err
    }
    refresh := randomHex(32)
    if _, err := t.DB.ExecContext(ctx, `
        INSERT INTO refresh_tokens (id, user_id, created_at, expires_at) VALUES (?, ?, ?, ?),
        digestSecret(t.secret, refresh), userID, now.UTC(), now.Add(t.refreshTTL).UTC()); err != nil {
        return TokenPair{}, fmt.Errorf("写入刷新令牌失败: %w", err)
    }
    return TokenPair{
        AccessToken: access,
        TokenType:   "Bearer",
        ExpiresIn:   int(t.accessTTL.Seconds()),
        RefreshToken: refresh,
        ExpiresAt:   time.Unix(claims.ExpiresAt, 0),
    }, nil
}
```

接口层 POST /api/token 与 Cookie

方案共用同一套口令校验、失败文案与限流逻辑：锁定期的响应与凭据错误完全一致，不使用 429 暴露账号是否存在。

2.4 校验：一个入口，两种凭据

受保护接口统一经过 RequireAuth：先看 Authorization: Bearer，再退回到 Cookie

会话；两条分支最终都调用 WithSession

把同一份身份放进请求上下文。业务代码因此完全不需要知道调用方用的是哪种方案。

代码 2-3 解析 Bearer 头 (backend/internal/httpx/httpx.go)

```
func bearerToken(r *http.Request) (string, bool) {
    header := r.Header.Get("Authorization")
    const prefix = "Bearer "
    if len(header) <= len(prefix) || !strings.EqualFold(header[:len(prefix)], prefix) {
        return "", false
    }
    return strings.TrimSpace(header[len(prefix):]), true
}
```

```

func RequireAuth(next http.Handler, sessions *auth.SessionStore, tokens *auth.TokenService) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // 1) Authorization: Bearer <jwt>
        if raw, ok := bearerToken(r); ok {
            claims, err := tokens.Verify(raw)
            if err != nil {
                Fail(w, http.StatusUnauthorized, "unauthorized")
                return
            }
            session, err := sessions.ByUserID(r.Context(), claims.Subject)
            switch {
            case errors.Is(err, auth.ErrNoSession):
                Fail(w, http.StatusUnauthorized, "unauthorized")
                return
            case err != nil:
                slog.Error("读取用户失败", "error", err)
                PermanentFailure(w)
                return
            }
            next.ServeHTTP(w, r.WithContext(WithSession(r.Context(), session)))
            return
        }

        // 2) Cookie：服务端会话
        cookie, err := r.Cookie(CookieName)
        if err != nil {
            Fail(w, http.StatusUnauthorized, "unauthorized")
            return
        }
        session, err := sessions.Lookup(r.Context(), cookie.Value)
        switch {
        case errors.Is(err, auth.ErrNoSession):
            Fail(w, http.StatusUnauthorized, "unauthorized")
            return
        case err != nil:
            // 会话存储不可用（例如数据库故障）：这是"服务不可用"，
            // 不能把已登录用户误判成会话失效，否则客户端会去重新登录而问题依旧。
            slog.Error("读取会话失败", "error", err)
            PermanentFailure(w)
            return
        }
        next.ServeHTTP(w, r.WithContext(WithSession(r.Context(), session)))
    })
}

```

注意其中的关键一步：校验令牌只得到 `sub`，随后 `ByUserID` 回 `users` 表取 `role` 与 `class_id`。这正是「令牌不承载权限」的落点——即便有人拿到令牌，也无法通过改载荷提权（改了签名就不对）。

2.5 刷新、轮换与撤销

刷新令牌是一次性的：使用后立即删除并签发新的一对（轮换），旧刷新令牌随即失效。签名校验用 `hmac.Equal` 做常量时间比较，避免时序侧信道。

代码 2-5 校验签名与有效期 (backend/internal/auth/token.go)

```
func (t *TokenService) Verify(token string) (TokenClaims, error) {
    parts := strings.Split(token, ".")
    if len(parts) != 3 {
        return TokenClaims{}, ErrInvalidToken
    }
    signing := parts[0] + "." + parts[1]
    mac := hmac.New(sha256.New, t.secret)
    mac.Write([]byte(signing))
    expected := base64.RawURLEncoding.EncodeToString(mac.Sum(nil))
    if !hmac.Equal([]byte(expected), []byte(parts[2])) {
        return TokenClaims{}, ErrInvalidToken
    }
    payload, err := base64.RawURLEncoding.DecodeString(parts[1])
    if err != nil {
        return TokenClaims{}, ErrInvalidToken
    }
    var claims TokenClaims
    if err := json.Unmarshal(payload, &claims); err != nil {
        return TokenClaims{}, ErrInvalidToken
    }
    if claims.Subject <= 0 || t.now().Unix() >= claims.ExpiresAt {
        return TokenClaims{}, ErrInvalidToken
    }
    return claims, nil
}
```

代码 2-6 刷新令牌表：只存哈希，可轮换可撤销 (backend/internal/db/migrations/003_refresh_tokens.sql)

```
CREATE TABLE IF NOT EXISTS refresh_tokens (
```

2.6 安全要点

关注点	做法	代码位置
令牌被篡改	HS256 签名校验，常量时间比较；任一字符变化即 401	auth/token.go · Verify
密钥泄露/缺失	JWT_SECRET 来自环境变量，缺失时进程以非零状态退出，不落内置默认值	config/config.go
权限提权	载荷不含 role/class_id，每次请求回库取身份	httpx/httpx.go · RequireAuth
重放与长期有效	访问令牌 15 分钟；刷新令牌一次性轮换、可撤销	auth/token.go · Issue/Refresh
账号枚举	未知用户也执行等成本假校验；失败文案与锁定期响应一致	server/server.go · tokenLogin
暴力破解	按「用户名 + 客户端 IP」限流	auth/ratelimit.go
令牌落盘风险	仓库不含真实密钥（.env 被 gitignore），密钥不下发浏览器	.gitignore / .env.example

2.7 与 Cookie 会话方案的对照

CampusClaw · token 方案实现登录认证（浏览器实测）

本页所有结果都来自浏览器对真实接口的调用，未做任何改写。令牌只保存在内存里，通过 Authorization: Bearer 头发送。

student_a1

1. 换取访问令牌

2. 带 Bearer 取本班材料

3. 不带令牌

4. 篡改令牌

5. 学生令牌上传

6. 刷新令牌

7. 旧刷新令牌再用

令牌内容（JWT 是三段式：header.payload.signature）

token_type: Bearer expires_in: 900 秒
access_token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ1IiwiaWF0IjY3ZmIwZg5NzA...
refresh_token: 9cd5c3b27f00c90897f4b0f3c0685966...（服务端只存它的哈希，可撤销）

```
// header (算法与类型)
{
  "alg": "HS256",
  "typ": "JWT"
}

// payload (只有 sub 与时间戳，没有 role / class_id)
{
  "sub": 2,
  "jti": "67fb078970886166f9f480801a6cbf89",
  "iat": 1790738803,
  "exp": 1790739703
}

// 过期时间: 2026/9/30 11:41:43
```

请求结果

用学生令牌调用教师专属上传
HTTP 状态: **403**
令牌里没有角色声明，服务端按 sub 回库取角色 → 学生 → 拒绝

响应摘要

```
{
  "error": "forbidden"
}
```

实测截图 用学生令牌调用教师专属上传返回 403：令牌里没有角色声明，服务端按 sub 回库取角色判定

3. 基于知识库的问答

问答链路分两段：入库侧把材料正文切成带字符区间的切片并计算向量；查询侧先在本班范围内检索，取得切片之后才生成回答，并让每条结论都能指回原文。

3.1 总体链路

代码 3-1 两条链路总览（示意）

```
入库侧：上传 → 校验（白名单/大小/UTF-8） → 落盘 → 同一事务写入
materials + knowledge_entries + kb_chunks（切片 + 向量）

查询侧：提问（只用最新这一句）
→ 本班检索（class_id 取自会话）
→ 有切片？否 → 返回「资料中未找到相关内容」，不调用生成模型
是 → 组装（材料标题 + 切片序号 + 切片正文 + 编号）
→ 生成模型 → 回答中用 [1][2] 标注 → 返回 citations
```

3.2 入库：一次事务写三张表

材料、正文与切片必须同生共死：只写材料会让检索拿不到数据，只写切片则会让下载指向不存在的文件。因此切片与向量在同一事务内写入，失败整体回滚并删除已落盘的文件。

代码 3-2 切片与向量写入事务（backend/internal/kb/index.go）

```
func (ix *Indexer) WriteTx(ctx context.Context, tx *sql.Tx, materialID, classID int64, content string) error {
    if _, err := tx.ExecContext(ctx, `DELETE FROM kb_chunks WHERE material_id = ?`, materialID); err != nil {
        return fmt.Errorf("清理旧索引失败: %w", err)
    }
    chunks := Split(content, ix.ChunkSize, ix.Overlap)
    for _, chunk := range chunks {
        vector := ix.Embedder.Embed(chunk.Content)
        if _, err := tx.ExecContext(ctx, `
            INSERT INTO kb_chunks
            (material_id, class_id, chunk_index, start_offset, end_offset, content, embedding, embedding
            _dim)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)`,
            materialID, classID, chunk.Index, chunk.Start, chunk.End,
            chunk.Content, EncodeVector(vector), ix.Embedder.Dim(),
        ); err != nil {
            return fmt.Errorf("写入索引块失败: %w", err)
        }
    }
    return nil
}
```

3.3 分块与溯源

切片是检索与溯源的最小单位。分块策略是「Markdown 标题优先断开，过长的段落再按定长窗口切分并保留重叠」，每个切片都记录它在正文中的字符区间——这是后面能核对出处的前提。

代码 3-3 切片结构：文本 + 序号 + 字符区间（backend/internal/kb/chunk.go）

```
type Chunk struct {
    Index    int
    Start    int
    End      int
    Content  string
}
```

开发中在这里发现过一个真实缺陷：早期「合并过短段落」的逻辑会直接跳过两个标题之间的正文，导致长文档只索引到开头的一小段（表现为检索永远命中开头）。修复为「延长上一段的结束位置」，并补了回归测试 `TestSplitKeepsContentWithDenseHeadings`，断言块区间的并集覆盖全文。

3.4 向量化与检索

每个切片计算一个定长向量（字符 n-gram 经哈希技巧映射 + L2 归一化），检索用余弦相似度排序。由于向量已归一化，余弦退化为点积，计算足够快；同一输入必得同一向量，因此排序可复现、可判定。

代码 3-4 确定性嵌入 (backend/internal/kb/embed.go)

```
func (h *HashingEmbedder) Embed(text string) []float32 {
    vec := make([]float32, h.dim)
    for _, token := range Tokenize(text) {
        sum := fnv.New64a()
        _, _ = sum.Write([]byte(token))
        value := sum.Sum64()
        bucket := int(value % uint64(h.dim))
        sign := float32(1)
        if value&(1<<63) != 0 {
            sign = -1
        }
        vec[bucket] += sign
    }
    return Normalize(vec)
}
```

班级隔离有两条防线：候选集在 SQL

层就按会话班级过滤（跨班数据从不进入候选集），应用层再做一次断言。即便有人改坏了 SQL 条件，跨班切片也不会被返回。

代码 3-5 检索入口：班级只来自会话 (backend/internal/search/search.go)

```
func (s *Service) Search(ctx context.Context, classID int64, query string, topK int) ([]Result, int, error) {
    {
        trimmed := strings.TrimSpace(query)
        if trimmed == "" {
            return nil, 0, fmt.Errorf("%w: 查询不能为空", ErrInvalidQuery)
        }
        if len([]rune(trimmed)) > s.QueryMax {
            return nil, 0, fmt.Errorf("%w: 查询长度超过上限 %d", ErrInvalidQuery, s.QueryMax)
        }
        if topK <= 0 {
            topK = 5
        }
        if topK > s.TopKMax {
            return nil, 0, fmt.Errorf("%w: top_k 超过上限 %d", ErrInvalidQuery, s.TopKMax)
        }
    }

    rows, err := s.DB.QueryContext(ctx, `
        SELECT c.material_id, c.class_id, c.chunk_index, c.start_offset, c.end_offset,
               c.content, c.embedding, m.title, m.original_name
        FROM kb_chunks c
        JOIN materials m ON m.id = c.material_id
        WHERE c.class_id = ?
        ORDER BY c.material_id, c.chunk_index`, classID)
    // ... (此处省略，完整实现见仓库源码)
```

低于阈值的候选按「没有匹配」处理：哈希嵌入会有少量碰撞噪声，不设阈值时无关提问也会返回一堆低分结果。跨班内容既不进入候选集、也不体现在结果条数与错误信息中，对外表现为 200 + 空列表（不是 403/404，避免暗示「该资料属于其他班级」）。

3.5 先检索，再生成

问答服务只做两件事：调用检索拿到切片；有切片才调用生成模型。没有命中切片时完全不进入生成环节——避免模型凭参数记忆硬答天气、比分一类问题。

代码 3-6 先检索再生成；无命中直接返回固定文案（backend/internal/answer/answer.go）

```
func (s *Service) Ask(ctx context.Context, classID int64, question string, topK int) (Result, error) {
    if topK <= 0 || topK > MaxContext {
        topK = MaxContext
    }
    hits, _, err := s.Search.Search(ctx, classID, question, topK)
    if err != nil {
        return Result{}, err
    }

    // 无命中：不调用生成模型，直接用固定文案返回（课件明确要求）。
    if len(hits) == 0 {
        return Result{
            Answer:    NoEvidenceText,
            Citations: []Citation{},
            ModelCalled: false,
            Engine:    "none",
        }, nil
    }

    citations := make([]Citation, 0, len(hits))
    for i, hit := range hits {
        citations = append(citations, Citation{
            Index:      i + 1,
            MaterialID: hit.MaterialID,
            MaterialTitle: hit.MaterialTitle,
            OriginalName: hit.OriginalName,
        })
    }
    // ...（此处省略，完整实现见仓库源码）
}
```

交给模型的内容由服务端组装，客户端注入不了 system 消息：

代码 3-7 提示词组装：只有标题、切片序号、切片正文与提问（backend/internal/answer/answer.go）

```
func BuildPrompt(question string, chunks []search.Result) (system string, user string) {
    var b strings.Builder
    fmt.Fprintf(&b, "问题： %s\n\n本班材料片段：\n", question)
    for i, c := range chunks {
        fmt.Fprintf(&b, "\n[%d] 《%s》 （第 %d 块，字符 %d-%d) \n%s\n",
            i+1, c.MaterialTitle, c.ChunkIndex+1, c.StartOffset, c.EndOffset, strings.TrimSpace(c.Content))
    }
    b.WriteString("\n请只依据以上片段作答，不要使用片段之外的知识；每个结论后用 [编号] 标注出处；片段不足以回答时直接说明未找到。")
    return "你是 CampusClaw 的教研助手，只能依据给定的本班材料片段作答。", b.String()
}
```

3.6 引用标注与出处

返回体里的 citations[i].index 与回答中的 [i] 严格对应，每条出处包含材料 id、标题、原始文件名、切片序号、字符区间与摘录。前端点击出处可回到材料详情并高亮该字符区间；摘录取自 MySQL 的切片正文，而不是向量库的载荷。

字段	含义	用途
material_id / material_title	来源材料	点击可打开详情（仍受班级隔离约束）
chunk_index	切片序号	定位到第几块
start_offset / end_offset	在正文中的字符区间	按区间截取正文即得到片段，可逐字符核对
snippet	切片正文摘录	直接展示依据，无需二次请求
score	相似度	用于展示相关度；低于阈值的候选不返回

3.7 生成模型接入

生成这一步做成接口 `ChatClient`：配置了网关密钥就用真实模型（DeepSeek，兼容 OpenAI 的 `/chat/completions`），未配置则退回本地摘录实现（内容全部来自检索结果，同样可溯源），便于离线复现。返回体的 `engine` 会标明本次由谁生成。

代码 3-8 调用对话网关（`backend/internal/answer/gateway.go`）

```
func (g *GatewayClient) Answer(ctx context.Context, question string, chunks []search.Result) (string, error) {
    {
        system, user := BuildPrompt(question, chunks)
        payload, err := json.Marshal(chatRequest{
            Model: g.Model,
            Messages: []chatMessage{
                {Role: "system", Content: system},
                {Role: "user", Content: user},
            },
            // 温度压低：本课要的是"依据片段回答"，不是创作。
            Temperature: 0.2,
            MaxTokens: 500,
            Stream:      false,
        })
        if err != nil {
            return "", err
        }

        req, err := http.NewRequestWithContext(ctx, http.MethodPost, g.BaseURL+"/chat/completions", bytes.NewReader(payload))
        if err != nil {
            return "", err
        }
        req.Header.Set("Content-Type", "application/json")
        req.Header.Set("Authorization", "Bearer "+g.APIKey)

        resp, err := g.HTTP.Do(req)
        if err != nil {
            return "", fmt.Errorf("调用对话网关失败: %w", err)
        }
        defer resp.Body.Close()

        body, err := io.ReadAll(io.LimitReader(resp.Body, 1<<20))
        if err != nil {
            return "", fmt.Errorf("读取网关响应失败: %w", err)
        }
        // ...（此处省略，完整实现见仓库源码）
    }
}
```

3.8 实测结果

用例	结果
问本班材料里有依据的问题	200，answer 带 [1][2] 标注，citations 2 条，engine=gateway:deepseek-chat
问本班材料没有依据的问题（天气）	200，answer=「资料中未找到相关内容」，citations 为空，model_called=false
未登录提问	401，不返回任何回答或切片
跨班提问	200 + 空结果（不暗示资料归属）
切片区间一致性	按 start/end 截取正文与返回片段逐字符一致

CampusClaw · 基于知识库的 AI 问答（浏览器实测）

先在本班材料检索，取得切片之后才生成回答；回答中的 [1][2] 与下方出处列表一一对应。以下结果均来自真实接口调用。

导入环节为什么用情境图？

提问

问一个本班没有依据的问题（天气）

未登录提问

回答

HTTP 200 model_called: true engine: gateway:deepseek-chat

依据本班材料片段，导入环节用情境图的原因如下：

– 用一张生活情境图引发预测，目的是把学生的注意力拉到「内容会怎么发展」上，为后面的概括训练做铺垫。[1]

– 导入环节由情境图入题，先让学生预测内容。[2]

补充说明：片段中只提到“情境图”用于引发预测，聚焦内容发展并为概括训练铺垫，未进一步说明其他原因。

生成模型的输入只有：材料标题、切片序号、切片正文、用户提问，没有向量分毫，也没有其他信息的切片。

出处 (citations)

[1] A班·教研材料示例：语文阅读课教学设计

出处文件：A班-语文阅读课教学设计.md，第 2 块，字符 320-588，相似度 0.282
导入环节的设计意图 导入环节用一张生活情境图引发预测，目的是把学生的注意力拉到「内容会怎么发展」上，为后面的概括训练做铺垫。教师只提问不评价，先收集学生的不同预测，再在精读环节逐一验证。## 分型作业设计 基础层：用一句话概括全文；提高层：找出两处关键词并说明理由；挑战层：仿写一段细节描写。## 教学反思要点 精读时间容易被个别学生的发言拖长，需要提前约定「一句话回答」的规则；概括能力的评价标准要提前给学生，避免课堂上临时加码。> 本条属于 A 班，B 班账号不应读到此；用于演示班级隔离与内容屏蔽。

[2] A班·教研材料示例：语文阅读课教学设计

出处文件：A班-语文阅读课教学设计.md，第 1 块，字符 0-320，相似度 0.225
A班 语文阅读课教学设计 本设计面向五年级语文阅读单元，整节课 40 分钟，重点是训练概括能力与语言积累。## 教学目标 – 能用一句话概括段落主要内容，说清楚「谁、做了什么、结果如何」。– 能找出文中的关键词，并说明它为什么关键。– 能在你写中迁移课课的设计意图\n\n导入环节用一张生活情境图引发预测，目的是把学生的注意力拉到「内容会怎么发展」上，\n\n为后面的概括训练做铺垫。教师只提问不评价，先收集学生的不同预测，再在精读环节逐一验证。## 分型作业设计 基础层：用一句话概括全文；提高层：找出两处关键词并说明理由；挑战层：仿写一段细节描写。## 教学反思要点 精读时间容易被个别学生的发言拖长，需要提前约定「一句话回答」的规则；概括能力的评价标准要提前给学生，避免课堂上临时加码。> 本条属于 A 班，B 班账号不应读到此；用于演示班级隔离与内容屏蔽。

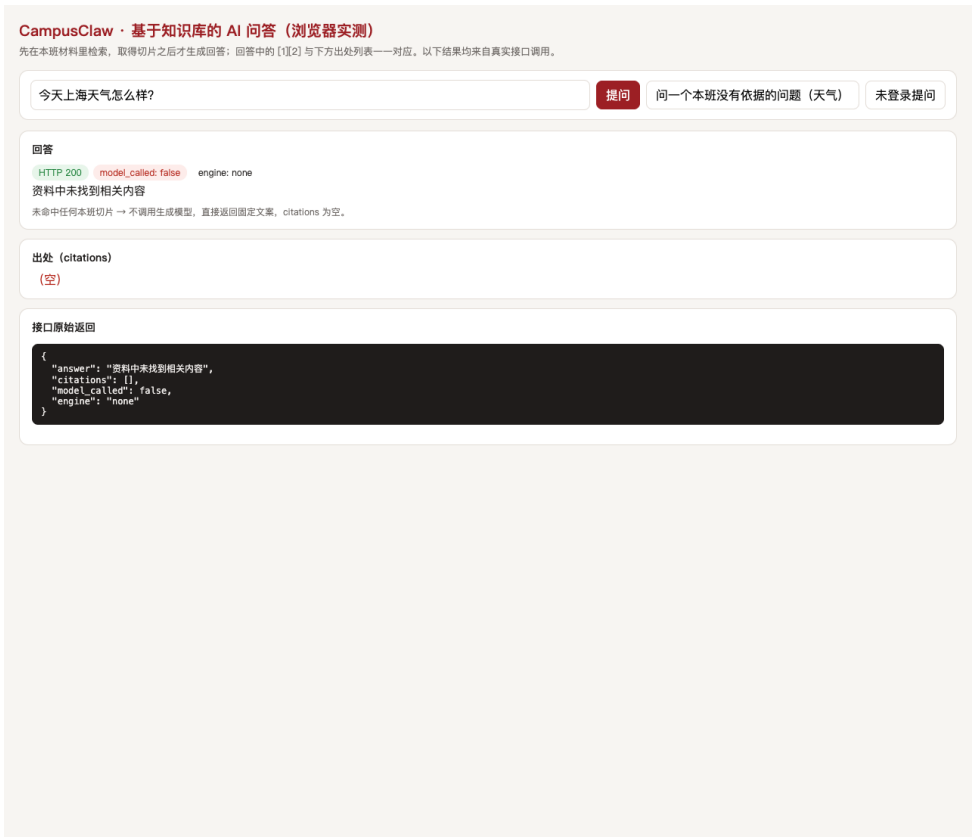
接口原始返回

```
{  "answer": "依据本班材料片段，导入环节用情境图的原因如下：\n\n– 用一张生活情境图引发预测，目的是把学生的注意力拉到「内容会怎么发展」上，为后面的概括训练做铺垫。[1]\n\n– 导入环节由情境图入题，先让学生预测内容。[2]\n\n补充说明：片段中只提到“情境图”用于引发预测，聚焦内容发展并为概括训练铺垫，未进一步说明其他原因。",  "citations": [    {      "index": 1,      "material_id": 23,      "material_title": "A班·教研材料示例：语文阅读课教学设计",      "original_name": "A班-语文阅读课教学设计.md",      "chunk_index": 1,      "start_offset": 320,      "end_offset": 588,      "snippet": "## 导入环节的设计意图\n\n导入环节用一张生活情境图引发预测，目的是把学生的注意力拉到「内容会怎么发展」上，\n\n为后面的概括训练做铺垫。教师只提问不评价，先收集学生的不同预测，再在精读环节逐一验证。## 分型作业设计 基础层：用一句话概括全文；提高层：找出两处关键词并说明理由；挑战层：仿写一段细节描写。## 教学反思要点 精读时间容易被个别学生的发言拖长，需要提前约定「一句话回答」的规则；概括能力的评价标准要提前给学生，避免课堂上临时加码。> 本条属于 A 班，B 班账号不应读到此；用于演示班级隔离与内容屏蔽。",      "score": 0.28194641783961883    },    {      "index": 2,      "material_id": 23,      "material_title": "A班·教研材料示例：语文阅读课教学设计",      "original_name": "A班-语文阅读课教学设计.md",      "chunk_index": 0,      "start_offset": 0,      "end_offset": 320,      "snippet": "# A班 语文阅读课教学设计 本设计面向五年级语文阅读单元，整节课 40 分钟，重点是训练概括能力与语言积累。",      "score": 0.22546877834178813    }  ],  "model_called": true,  "engine": "gateway:deepseek-chat"}
```

实测截图 真实模型回答：每条结论后用 [1][2] 标注，下方列出出处（材料标题、块序号、字符区间、摘录）

CampusClaw · 身份认证与知识库问答实现方案

第 13 页

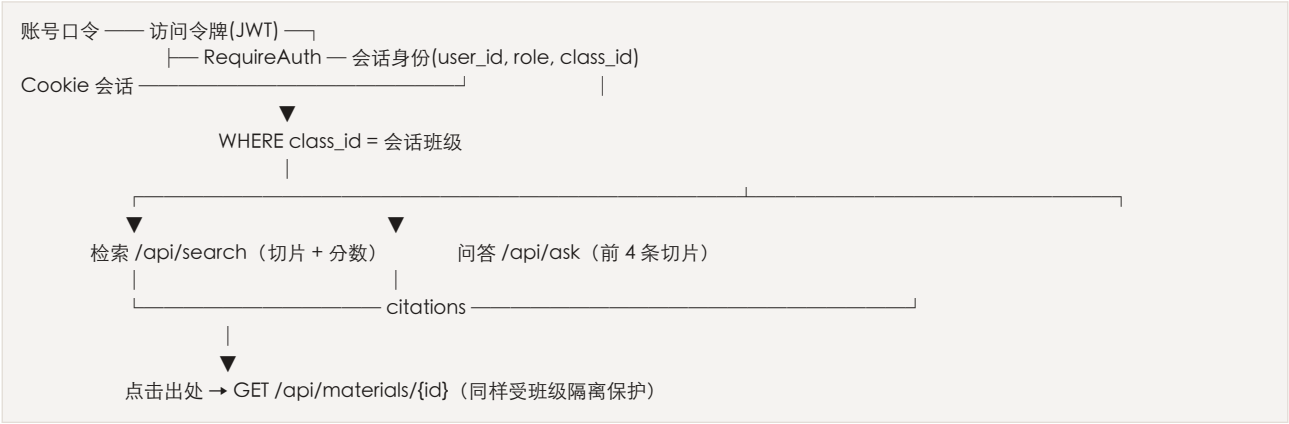


实测截图 问「今天上海天气怎么样」：本班无依据 → 不调用生成模型，返回固定文案，citations 为空

4. 两条链路的衔接

认证与问答并不是两套独立系统，它们在同一条链路上前后相接：认证决定你是谁、属于哪个班；问答用这个班级限定检索范围；检索结果又通过材料 id 回到同一个受鉴权保护的详情接口。

代码 4-1 认证 → 隔离 → 检索 → 引用 的衔接关系



5. 已知限制与后续

限制	影响	后续做法
访问令牌无状态	登出后令牌在有效期内（15 分钟）仍可用	需要立即失效时引入黑名单或缩短有效期
嵌入是词形相似	同义改写、跨语言检索会明显变弱	把 Embedder 换成真实语义模型（接口已预留）
单实例部署	登录限流在进程内存中，多副本会不一致	限流状态外置（Redis）后再扩容
知识库只支持纯文本	PDF / Word / 图片无法入库	后续迭代增加解析与 OCR
未实现流式输出与多轮上下文	回答一次返回，不保留对话历史	按课程安排在第 5 课实现
向量存于 MySQL	数据量增大后相似度计算会变慢	数据量上来后迁移到向量库（Qdrant 等）

附录 A 代码索引

模块	文件	关键函数
令牌签发与校验	backend/internal/auth/token.go	Issue / Verify / Refresh / Revoke
会话存储	backend/internal/auth/session.go	Create / Lookup / ByUserID
登录限流	backend/internal/auth/ratelimit.go	Allowed / Fail / Reset
统一鉴权入口	backend/internal/httpx/httpx.go	RequireAuth / bearerToken / SetSessionCookie
token 接口	backend/internal/server/server.go	tokenLogin / tokenRefresh / tokenLogout
分块与向量	backend/internal/kb/chunk.go · embed.go · index.go	Split / Embed / WriteTx / RebuildAll
检索	backend/internal/search/search.go	Search（班级过滤 + 相似度排序）
问答	backend/internal/answer/answer.go · gateway.go	Ask / BuildPrompt / GatewayClient.Answer
入库事务	backend/internal/materials/store.go	Create（三表同事务）
数据模型	backend/internal/db/migrations/*.sql	users / sessions / refresh_tokens / materials / knowledge_entries / kb_chunks

附录 B 复现步骤

附录 B 从零复现与验证

```
# 1) 启动（Docker Compose，唯一标准启动方式）
cp .env.example .env      # 填 SESSION_SECRET、DB_*、SEED_*_PASSWORD、JWT_SECRET
docker compose up --build -d
# 打开 http://localhost:8080

# 2) token 方案
curl -s -X POST localhost:8080/api/token -H 'Content-Type: application/json' \
  -d '{"username":"student_a1","password":"<预置口令>"}'
curl -s -H "Authorization: Bearer <access_token>" localhost:8080/api/materials

# 3) 知识库问答
curl -s -X POST localhost:8080/api/ask -H 'Content-Type: application/json' \
  -H 'Cookie: campusclaw_session=<会话>' \
  -d '{"question":"导入环节为什么要用情境图？"}'

# 4) 验收脚本（逐条对照规约）
BASE_URL=http://localhost:8080 TEACHER_PW=... STUDENT_A_PW=... STUDENT_B_PW=... ./scripts/verify.sh
./scripts/verify-retrieval.sh
./scripts/eval-retrieval.sh      # 评测集：命中率 + 隔离用例
```

本文所有代码片段均从仓库源码按函数抽取，实测结果与截图取自本机真实运行（Nginx → Go → MySQL，回答生成接入 DeepSeek 网关）。仓库地址：<https://github.com/OhtoAi583/campusclaw>